데이터분석기법

13주차: Convolutional neural networks and represenation learning

양승진
slowmoyang@gmail.com

세종대학교

2025년 5월 27일

Goals for Today

- Understand the limitations of fully connected neural networks in computer vision
- Learn the core principles behind convolutional neural networks, including local connectivity and parameter sharing
- Explore the building blocks of CNNs: convolution, activation, pooling, normalization, and dropout
- Introduce representative CNN architectures
- Familiarize with key computer vision tasks

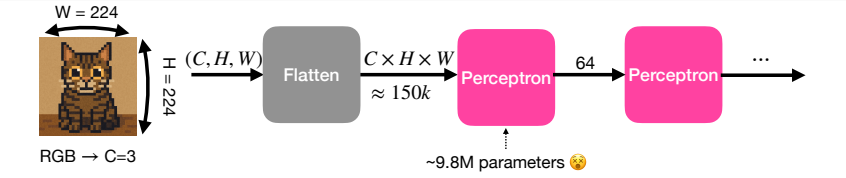
Computer Vision

Computer Vision

- Computer vision (CV) is a field of AI that enables machines to interpret and understand visual information from the world: 🤖 + 👀
- Traditional CV methods rely on hand-crafted features: Scale-invariant feature transform (SIFT), Histogram of oriented gradients (HOG), and etc.
- These features often failed under real-world variability

A pedestrian image (left) and its gradient (right). Source: [11]

DL for CV?



- Loss of spatial structure
  - 2D images are 3D arrays, but fully connected neural networks (FCNNs) require 1D input
  - Flattening destroys the spatial relationships between pixels
  - FCNNs treat every pixel independently, ignoring local correlations like edges
  - If a pattern (like an edge) shifts slightly in position, the FCNN sees it as a completely different input
- Parameter explosion
  - For high-resolution images, the number of parameters becomes massive
  - A single perceptron layer applied to a 3×224×224 image requires about 9.8 million parameters to produce a 64-dim hidden vector

Convolutional Neural Networks

Core Ideas

- **Convolutional Neural Networks (CNNs)** are a type of neural network designed to process data with a grid-like structure, such as 1D time-series or 2D images
- CNNs are particularly effective at capturing local patterns using **convolution** operations[1]
- **Local Connectivity**: convolution extracts local features from small subregions of the image
- **Parameter sharing**: convolution uses shared parameters to detect local features throughout the entire image

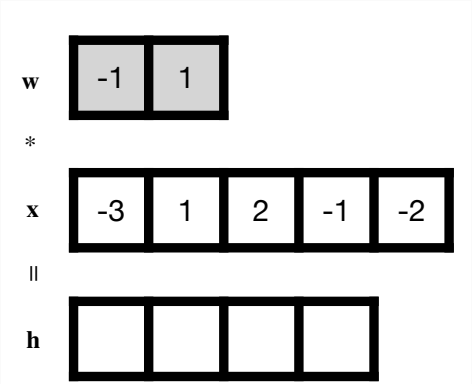
[1] Convolutional neural networks typically use *cross-correlation* instead of convolution, but since the kernels are learnable and the distinction is minor, we refer to cross-correlation as "convolution" throughout this lecture for simplicity.

1D Discrete Convolution

Convolution

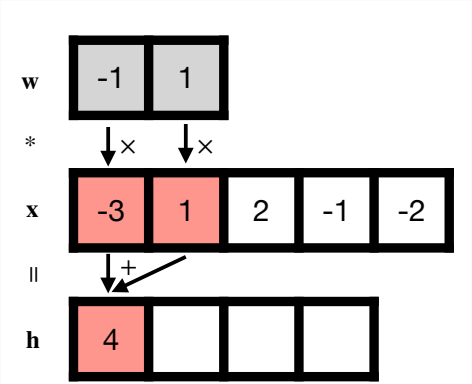
- **Input**, $\mathbf{x} \in \mathbb{R}^M$ (e.g. $\mathbf{x} = [x_1, x_2, x_3, x_4, x_5]$)
- **Kernel** or **filter**, $\mathbf{w} \in \mathbb{R}^N$ (e.g. $\mathbf{w} = [w_1, w_2]$)
- **Output (feature map)**, $\mathbf{w} * \mathbf{x}$ or $(\mathbf{w} * \mathbf{x})_i = \sum_{n=1}^{N} w_n x_{i+n}$
  - e.g. $\mathbf{w} * \mathbf{x} = (w_0 x_0 + w_1 x_1, w_0 x_1 + w_1 x_2, w_0 x_2 + w_1 x_3, w_0 x_3 + w_1 x_4)$

Visualization of 1D Discrete Convolution



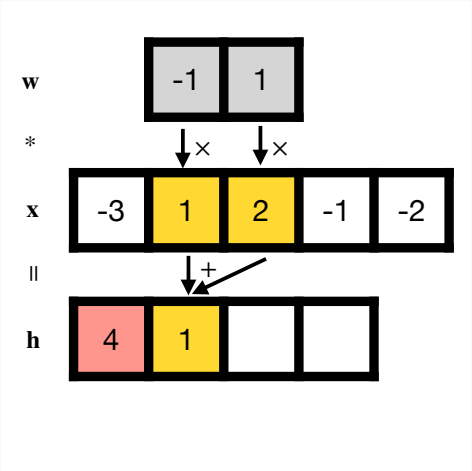
- Kernel: **w** / Input: **x** / Output feature map: **h**

Visualization of 1D Discrete Convolution (cont.)



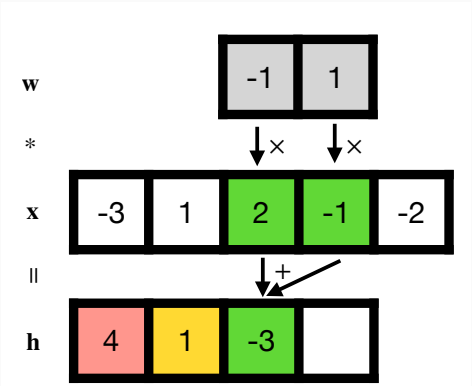
- Kernel: **w** / Input: **x** / Output feature map: **h**

Visualization of 1D Discrete Convolution (cont.)



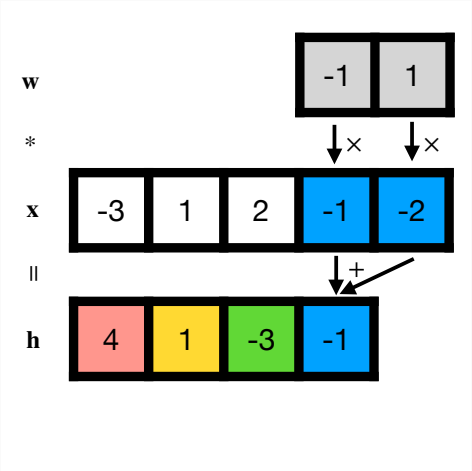
- Kernel: **w** / Input: **x** / Output feature map: **h**

Visualization of 1D Discrete Convolution (cont.)



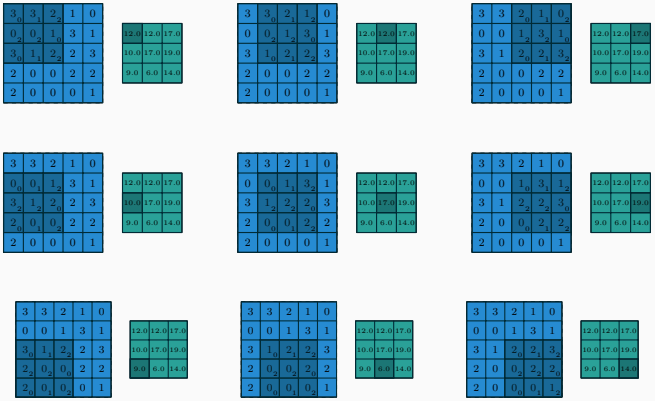
- Kernel: **w** / Input: **x** / Output feature map: **h**

1D Discrete Convolution (cont.)



- Kernel: **w** / Input: **x** / Output feature map: **h**

2D Discrete Convolution (cont.)

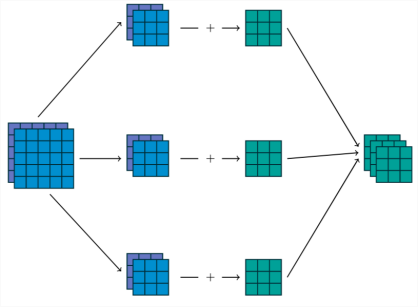


Source: [1]

- 2D discrete convolution: $(W * X)_{ij} = \sum_h \sum_w W_{h,w} X_{i+h,j+w}$

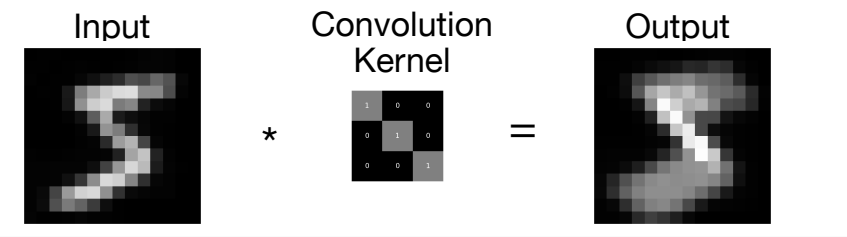
Multi-Channel Convolution

- Input images can have multiple channels, such as RGBA images with four channels: red, green, blue, and alpha (transparency)
- A convolution takes a multi-channel input (leftmost stack of purple-blue squares), and applies multiple kernels (small purple-blue squares)
- In each path, each kernel processes the corresponding input channel, and the results are summed element-wise to produce one channel of the output feature map
- Combining multiple such outputs results in a multi-channel output (rightmost stack of green squares)
- This enables the construction of deeper and more expressive models



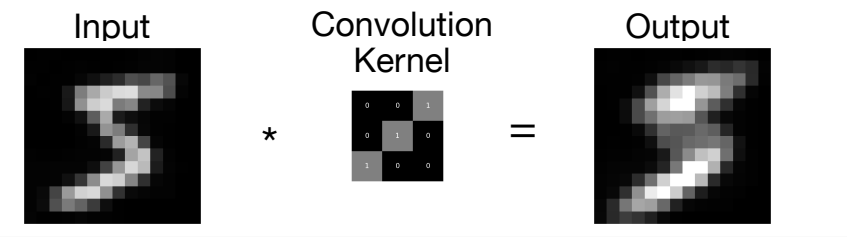
Source: [1]

Convolution as Edge Detection



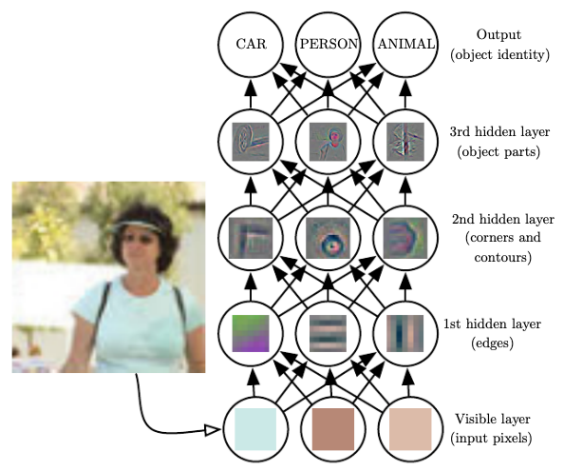
- **Input** is a grayscale image of the digit 5, represented as a 2D array of pixel intensities, where brighter pixels indicate higher intensity, and darker pixels indicate lower intensity
- **Convolution kernel** is a small 3×3 matrix that slides over the input image to detect edges
- **Output feature map** is the result of applying the kernel to the input image, highlighting edges where intensity changes align with the kernel's pattern

Convolution as Edge Detection (cont.)



- **Input** is a grayscale image of the digit 5, represented as a 2D array of pixel intensities, where brighter pixels indicate higher intensity, and darker pixels indicate lower intensity
- **Convolution kernel** is a small 3×3 matrix that slides over the input image to detect edges
- **Output feature map** is the result of applying the kernel to the input image, highlighting edges where intensity changes align with the kernel's pattern

Convolution as Edge Detection (cont.)

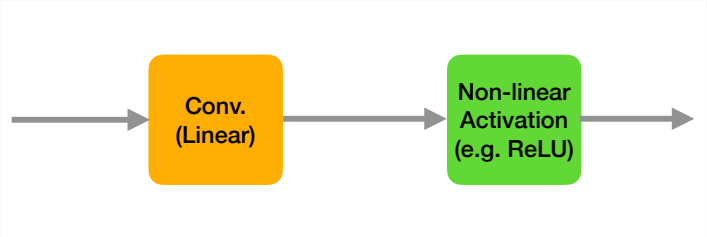


Source: [13, 3]

Convolution Arithmetic

To gain a deeper understanding of convolution, refer to the following link:
https://github.com/vdumoulin/conv_arithmetic

Linearity of Convolution

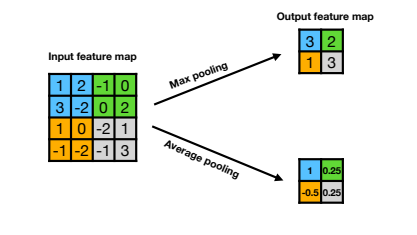


- Convolution is a linear operation

$$\mathbf{w} * (a\mathbf{u} + b\mathbf{v}) = a\mathbf{w} * \mathbf{u} + b\mathbf{w} * \mathbf{v} \tag{1}$$

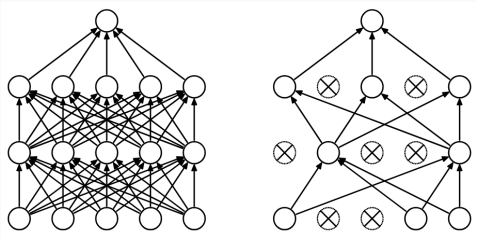
- In convolutional neural networks (CNNs), a convolution layer is typically followed by a non-linear activation function, just like in fully connected neural networks

Pooling



- Pooling is a downsampling operation that reduces the spatial dimensions of feature maps, making the model more translation invariant and reduces computational cost
- A pooling window (e.g., 2×2) slides across the input feature map
- For each window, it applies an aggregation function, such as max and average
- PyTorch provides a bunch of pooling methods (link)

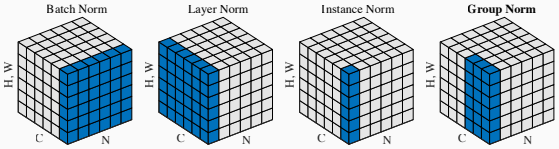
Dropout



Left: Standard Neural Net. Right: After applying dropout. Source: [9]

- Dropout is a regularization technique where, during training, a random subset of neurons is ”dropped” (set to zero) in each layer, preventing overfitting by encouraging the network to learn redundant representations
- Undropped nodes are scaled up by $\frac{1}{1-p_{\text{dropout}}}$ during training to ensure the network's output remains consistent in scale
- See torch.nn.Dropout

Normalization



Source: [12]

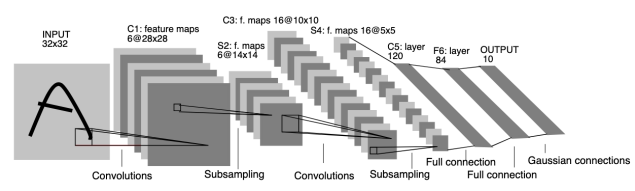
- **Normalization** is a technique used to improve the training of DL models by adjusting the inputs of each layer to have a mean of zero and a standard deviation of one, thereby stabilizing the learning process
- During training, a normalization layer uses the mean and standard deviation of each batch for backpropagation
- In the inference phase, a normalization layer uses a running average of the means and standard deviation from the training phase, maintaining stable outputs
- Batch normalization is widely used in DL models for image data
- See Normalization Layers in PyTorch

CNN Demo

ConvNetJS CIFAR-10 demo

Architectures

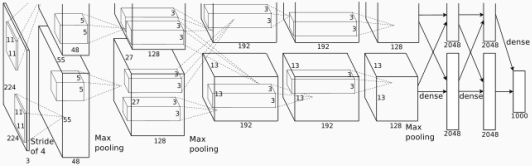
LeNet



Architecture of LeNe-5 [7]

- Paper: LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (2002). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278-2324.
- Task: Digit recognition (MNIST)
- Architecture: Alternating convolution and pooling layers followed by fully connected layers
- Key Idea: Introduced the basic CNN structure—convolution, pooling, and fully connected (FC) layers for image classification

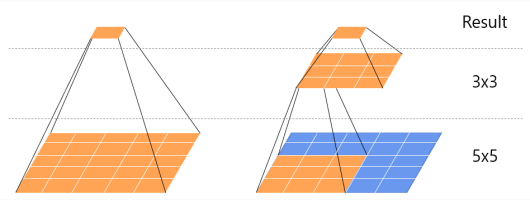
AlexNet



Architecture of AlexNet [6]

- Paper: Krizhevsky, A., Sutskever, I., Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. Advances in neural information processing systems, 25.
- Task: Image classification
- Architecture: 5 conv. + 3 FC layers, ReLU activation, dropout, and overlapping max pooling
- Key Idea: Demonstrated that deep CNNs can significantly outperform traditional methods on large datasets using GPU acceleration

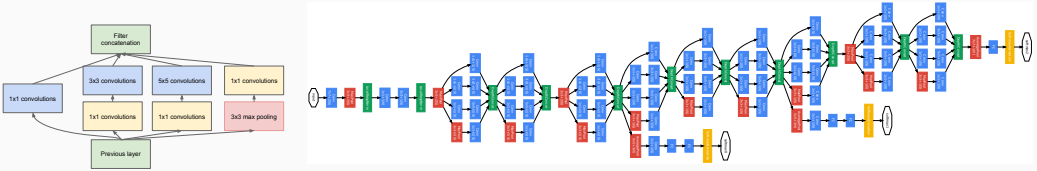
VGGNet



A comparison of the 5×5 and 3×3 convolution filters

- Paper: Simonyan, K., Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.
- Task: Image classification
- Architecture: Very deep network (up to 19 layers) using only 3×3 convolutions and 2×2 max pooling
  - one 5×5 conv.: $5 \times 5 = 25$ parameters / two 3×3 conv.: $2 \times 3 \times 3 = 18$ parameters
- Key Idea: Showed that depth and simplicity (stacking small filters) can improve performance

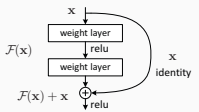
GoogLeNet

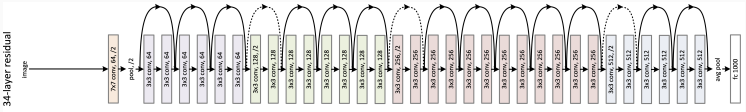


Source: [10]

- Paper: Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., et al. (2015). Going deeper with convolutions. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 1-9).
- Image classification
- Architecture: Introduced the Inception module, combining multiple filter sizes (1×1, 3×3, 5×5) and max pooling in parallel
- Key Idea: Efficiently captures multi-scale features while reducing parameters with 1×1 convolutions

ResNet

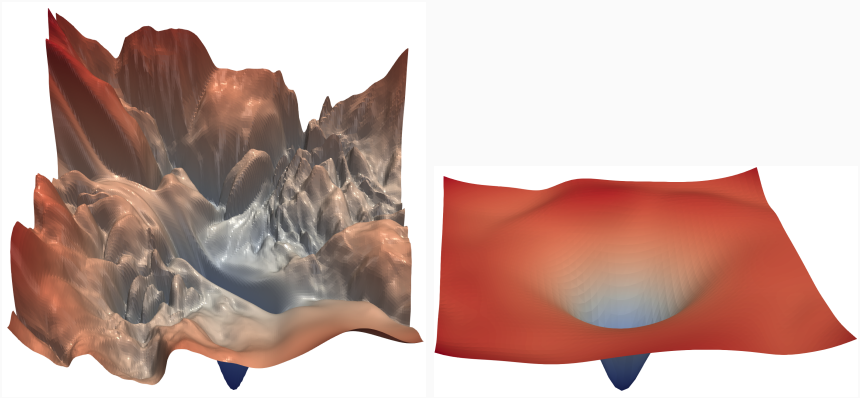




Source: [5]

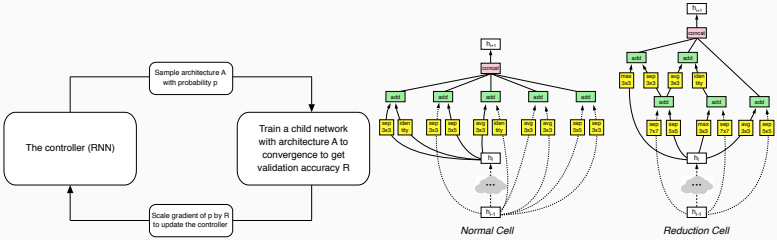
- Paper: He, K., Zhang, X., Ren, S., Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778).
- Task: Image Classification
- Key Idea: Enables training of very deep networks by addressing the vanishing gradient problem with residual connections
- Architecture: Very deep network (up to 152 layers) using residual connections

ResNet (cont.)



The loss surfaces of ResNet-56 (left) without and (right) with residual connection. Source: [8]

NASNet



Source: [14]

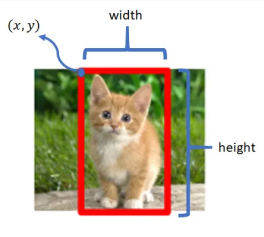
- Paper: Zoph, B., Vasudevan, V., Shlens, J., Le, Q. V. (2018). Learning transferable architectures for scalable image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 8697-8710).
- Task: Image classification
- Key Idea: Use Neural Architecture Search (NAS) with reinforcement learning to find the best convolutional cell structures

CNN Model Zoo

- Pretrained DL models for CV by PyTorch Teams

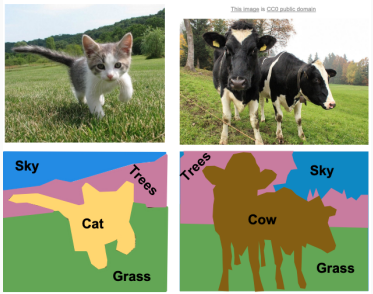
CV Tasks

Object Detection



- Goal: Both detect the presence of objects and predict their positions
  - $\rightarrow$ Classification + Regressing bounding box coordinates
- Example: Detect a cat in an image

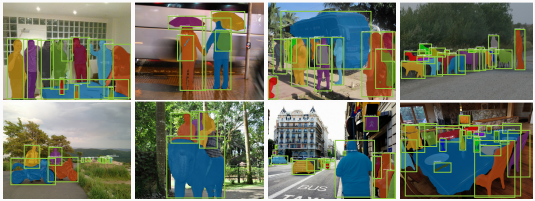
Semantic Segmentation



Source: [2]

- Goal: Assign a class label to each pixel in the image
  → pixel-wise classification
- Example: Identify which pixels belong to roads, pedestrians, cars, etc., in a self-driving car scenario
- Models: FCN, U-Net, DeepLab

Instance Segmentation



Source: [4]

- Segment each individual object instance, even if they are of the same class
  - → semantic segmentation + object detection
- Segment two different cats in the same image
- Popular Models: Mask R-CNN

Keypoint Detection

Source: [4]

- Goal: Detect specific points (e.g., joints) on objects or human bodies
- Example: Detect human joints like knees, elbows, etc.
- Popular Models: Mask R-CNN, OpenPose, HRNet

Summary

- CNNs address the limitations of FCNNs by preserving spatial structure and reducing parameters via local connectivity and weight sharing
- Convolution layers extract local features, often visualized as edge detectors
- CNNs power key CV tasks like classification, object detection, segmentation, and keypoint detection

Questions**?**

References i

V. Dumoulin and F. Visin.
A guide to convolution arithmetic for deep learning, 2018.

Fei-Fei Li and Justin Johnson and Serena Yeung.
Lecture 11: Detection and Segmentation.
https://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture11.pdf, 2017.
Accessed: 2025-05-27.

I. Goodfellow, Y. Bengio, and A. Courville.
***Deep Learning*.**
MIT Press, 2016.
http://www.deeplearningbook.org.

References ii

- K. He, G. Gkioxari, P. Dollár, and R. B. Girshick.
  Mask R-CNN.
  CoRR, abs/1703.06870, 2017.
- K. He, X. Zhang, S. Ren, and J. Sun.
  Deep residual learning for image recognition.
  CoRR, abs/1512.03385, 2015.
- A. Krizhevsky, I. Sutskever, and G. E. Hinton.
  Imagenet classification with deep convolutional neural networks.
  Advances in neural information processing systems, 25, 2012.

References iii

Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner.
Gradient-based learning applied to document recognition.
Proceedings of the IEEE, 86(11):2278–2324, 2002.

H. Li, Z. Xu, G. Taylor, and T. Goldstein.
Visualizing the loss landscape of neural nets.
CoRR, abs/1712.09913, 2017.

N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov.
Dropout: a simple way to prevent neural networks from overfitting.
The journal of machine learning research, 15(1):1929–1958, 2014.

C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. E. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich.
Going deeper with convolutions.
CoRR, abs/1409.4842, 2014.

Wikipedia.
Histogram of oriented gradients — Wikipedia, the free encyclopedia.
http://en.wikipedia.org/w/index.php?title=Histogram%20of%20oriented%20gradients&oldid=1280028145, 2025.
[Online; accessed 27-May-2025].

Y. Wu and K. He.
Group normalization.
CoRR, abs/1803.08494, 2018.

M. D. Zeiler and R. Fergus.
Visualizing and understanding convolutional networks.
CoRR, abs/1311.2901, 2013.

B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le.
Learning transferable architectures for scalable image recognition.
CoRR, abs/1707.07012, 2017.